A Lean Formalization of the Dyadic Rank–Support Converse and Its Relation to Published Proofs

Draft — October 21, 2026

Abstract

For a binary dyadic matrix of order 2^ℓ, rank $2^{\ell-1}$ forces the sum of the index vectors in its signature support to be nonzero. Gómez-Fonseca, Matthews, Morris, and Pllaha explicitly propose this converse in Remark 3.4 of their 2026 preprint. We present a Lean 4 proof for every positive dimension, using a triangular block decomposition and descent of the image–kernel equality. The mathematical conclusion follows from established representation theory: we give the translation to the flatness criterion of Friedlander and Pevtsova, identify the descent step as a characteristic-two instance of their commuting-nilpotent perturbation principle, and explain an alternative implication of Kustin–Şega–Vraciu's exact-zero-divisor theorem. The published representation-theoretic criterion appeared in April 2005, approximately 21 years before the explicit conjecture was posted on May 3, 2026. The contribution is the checked implementation, its elementary matrix exposition, and an explicit connection between the coding-theoretic question and the earlier representation-theoretic criterion. We claim neither a new mathematical theorem nor priority for the proof method or its formalization. The full inductive Lean source, pinned dependencies, verification records, and bounded prior-art search are supplied with the manuscript.

1 Origin of the question and scope of the contribution

The precise question addressed here comes from *Combinatorial Analysis of Dyadic and Quasi-Dyadic Codes* by Anthony Gómez-Fonseca, Gretchen L. Matthews, Kirsten D. Morris, and Tefjol Pllaha. Their Remark 3.4 explicitly proposes the full converse of a rank criterion for dyadic matrices [1, Remark 3.4]. We credit those four authors for this source formulation. The present theorem is the implication from half rank to a nonzero support-vector sum; we specify both the coefficient field and the meaning of support in Section 2.

Our investigation began with mathematics dissertations made available through the University of Nebraska–Lincoln's DigitalCommons: Meraiah Martinez's *On Dyadic Parity Check Codes and Their Generalizations* [2] and Kirsten Morris's *Analysis of Graph-based Decoders for Quantum Low Density Parity Check Codes* [3]. Both repository records identify Christine A. Kelley as first advisor. Following the related literature led to the explicit 2026 conjecture. This discovery provenance is distinct from attributing that exact conjecture to either dissertation. The antecedent paper by Martinez, Pllaha, and Kelley [4] is the work cited by Gómez-Fonseca et al. in their discussion of the forward criterion [1, Remark 3.4].

The formalization was developed before the literature comparison documented here was complete. Subsequent investigation identified the applicable flatness criterion in Friedlander–Pevtsova [5, Example 2.6], also stated by Benson [7, Section 1.4, p. 6]. Accordingly, this paper reports an

independently developed implementation of a consequence of existing theory, not a newly discovered theorem. Its main technical presentation is an elementary induction in matrix coordinates. Its scope as a formalization is exact: the matrix proof is checked in Lean; the group-algebra translation and the two literature comparisons are ordinary mathematical arguments in this manuscript, not additional Lean theorems.

The chronology is striking: Friedlander–Pevtsova's article appeared in April 2005 [5], whereas the explicit dyadic rank–support conjecture was posted on May 3, 2026 [1], approximately *21 years later*. This interval illustrates a barrier to recognizing applicable results across coding theory and representation theory: the conjecture is phrased in terms of dyadic matrix rank and signature support, while the earlier criterion concerns flatness and linear terms in an elementary abelian group algebra. Our contribution includes making that correspondence explicit and presenting it alongside a checked matrix proof. The dates alone do not establish that the correspondence was previously unknown, or why the conjecture's authors did not use it; we do not claim historical priority for connecting the two formulations. April 2005 is a verified published precedent, not an established earliest date for the underlying result.

The accompanying artifact [11] records a general proof, rather than a finite-dimensional search certificate. A bounded search found no prior matching Lean statement or complete proof in the searched sources. Section 7 states the coverage and limitations of that search; it does not establish that this is the first formalization.

2 Definitions and the formalized statement

Fix $\ell \in \mathbb{N}$, let $G_\ell = \mathbb{F}_2^\ell$ with its additive group structure, and put $N = 2^\ell$. A signature is a function $s : G_\ell \to \mathbb{F}_2$. Its dyadic matrix is

$$D_s(x, y) = s(x + y) \qquad (x, y \in G_\ell). \tag{1}$$

This is the binary signature presentation used in the source [1, Definition 2.1]. Define its parity and first moment by

$$p(s) = \sum_{x \in G_\ell} s(x) \in \mathbb{F}_2, \qquad m(s) = \sum_{x \in G_\ell} s(x)x = \sum_{x \in \mathrm{supp}(s)} x \in G_\ell. \tag{2}$$

The last equality uses $s(x) \in \{0, 1\}$. In particular, $m(s)$ is a sum of *index vectors*, not the scalar sum of the entries of s. Every rank in this paper is ordinary matrix rank over $\mathbb{F}_2$.

Theorem 2.1 (Rank–support converse)**.** *For $\ell > 0$ and any $s : G_\ell \to \mathbb{F}_2$,*

$$\operatorname{rank} D_s = 2^{\ell-1} \quad \Longrightarrow \quad m(s) \neq 0.$$

The source of the conjectured implication is Remark 3.4 of [1]; its known-theory interpretation is established in Section 4. No condition that the support be a subspace, coset, or set of size two is imposed. We do not assert the unrestricted reverse implication. For example, if $a \neq 0$ and s is the indicator of $\{a\}$, then $m(s) = a$ while D_s is an invertible permutation matrix. Parity matters when stating an equivalence.

Lean uses the type `Point ell := Fin ell → ZMod 2`. The principal signature statement is the equivalent doubled-rank form Here `supportSum` is the sum over the finite support in (2). The doubled-rank proposition is vacuous for $\ell = 0$, since twice an integer cannot equal one. The public theorem `dyadic_rank_support_half` assumes positive dimension and uses Lean's natural-number expression `2 ^ ell / 2`; it applies to any matrix satisfying (1), with signature recovered as its zero row. The definitions are in `Basic.lean`, and the source-to-matrix equivalence is proved there [11].

Listing 1: The general signature interface.

```
def RankSupportAt (ell : ℕ) : Prop :=
  ∀ s : Signature ell,
    2 * (dyadicMatrix s).rank = 2 ^ ell → supportSum s ≠ 0

theorem rankSupportAt_all (ell : ℕ) : RankSupportAt ell
```

3 An elementary proof by dimension descent

We give the mathematical argument implemented in `Structural.lean` and `Recursive.lean` [11]. Dyadic squaring identities also occur in the source's preliminary properties [1, Proposition 2.2]; the calculations below make their role in this proof explicit.

Lemma 3.1. *For every signature s, $D_s^2 = p(s)I$. Any two dyadic matrices commute.*

Proof. The (x, y) entry of D_s^2 is $\sum_z s(x+z)s(z+y)$. If $x = y$, this equals $\sum_z s(x+z)^2 = p(s)$. If $x \neq y$, the involution $z \mapsto x + y + z$ has no fixed point and exchanges the two factors; paired summands cancel in characteristic two. For two signatures, the same change of variable interchanges their order in the convolution product. □

Definition 3.2. For an endomorphism C of an $\mathbb{F}_2$-vector space, write $\mathcal{E}(C)$ for the preimage property

$$Cv = 0 \quad \Longrightarrow \quad \text{there exists } w \text{ with } Cw = v.$$

If $C^2 = 0$, this property is equivalent to $\operatorname{im} C = \ker C$.

If $\ell > 0$ and D_s has half rank, Lemma 3.1 forces $p(s) = 0$: otherwise $D_s^2 = I$ would give full rank. In the square-zero case $\operatorname{im} D_s \subseteq \ker D_s$, so rank–nullity gives

$$2 \operatorname{rank} D_s = N \quad \Longleftrightarrow \quad \operatorname{im} D_s = \ker D_s. \tag{3}$$

It therefore suffices to show that $p(s) = 0 = m(s)$ is incompatible with $\mathcal{E}(D_s)$.

Lemma 3.3 (Triangular descent)**.** *Let B, C be commuting endomorphisms with $B^2 = 0$, and set*

$$T = \begin{pmatrix} C & B \\ 0 & C \end{pmatrix}.$$

If $\mathcal{E}(T)$ holds, then $\mathcal{E}(C)$ holds.

Proof. Take v with $Cv = 0$. Since $T(v, 0) = 0$, choose a, b such that

$$Ca + Bb = v, \qquad Cb = 0.$$

Apply the same property to $(b, 0)$, obtaining c, d with $Cc + Bd = b$ and $Cd = 0$. Then

$$Bb = B(Cc + Bd) = BCc = CBc, \qquad v = C(a + Bc).$$

Thus v has a preimage under C. No hypothesis $C^2 = 0$ is needed for this lemma itself. □

The Lean helper expresses the hypothesis using only the two coordinates required in the proof:

Listing 2: Checked preimage calculation from Recursive.lean.

```
theorem kernelInRange_descends {V : Type*} [AddCommGroup V] [Module F2 V]
    (C B : V →ₗ[F2] V) (hc : ∀ v, B (C v) = C (B v))
    (hb : ∀ v, B (B v) = 0)
    (h : ∀ v, C v = 0 → ∃ a b, C a + B b = v ∧ C b = 0) :
    KernelInRange C := by
  intro v hv
  obtain ⟨a, b, hab, hcb⟩ := h v hv
  obtain ⟨c, d, hcd, _⟩ := h b hcb
  have hbb : B b = B (C c) := by
    rw [← hcd, map_add, hb, add_zero]
  refine ⟨a + B c, ?_⟩
  rw [map_add, ← hc, ← hbb]
  exact hab
```

The application of this descent principle is a specialization of an existing published result; Section 4.1 gives the exact correspondence. We make no priority claim for the calculation.

For the inductive step, write $G_{\ell+1} = \mathbb{F}_2 \times G_\ell$ and let

$$a(x) = s(0, x), \qquad b(x) = s(1, x), \qquad t(x) = a(x) + b(x).$$

Set $A = D_a$, $B = D_b$, and $C = D_t = A + B$. Then

$$D_s = \begin{pmatrix} A & B \\ B & A \end{pmatrix}, \qquad PD_sP = \begin{pmatrix} C & B \\ 0 & C \end{pmatrix}, \qquad P = \begin{pmatrix} I & 0 \\ I & I \end{pmatrix} = P^{-1}. \tag{4}$$

Directly from (2),

$$p(t) = p(s), \qquad m(t) = \mathrm{tail}(m(s)), \qquad p(b) = m(s)_0. \tag{5}$$

Thus $p(s) = 0 = m(s)$ implies $p(t) = 0 = m(t)$ and $B^2 = 0$. Also $BC = CB$ by Lemma 3.1.

Lemma 3.4. *For every $\ell \geq 0$, if $p(s) = 0$ and $m(s) = 0$, then $\mathcal{E}(D_s)$ is false.*

Proof. Induct on ℓ. At $\ell = 0$ there is one index vector. Zero parity forces D_s to be the zero 1×1 matrix, so the nonzero vector 1 is killed but is not an image. At the successor step, suppose $\mathcal{E}(D_s)$ holds. The coordinate change in (4) preserves the preimage property. Lemma 3.3 gives $\mathcal{E}(C)$, whereas (5) makes t satisfy the induction hypothesis. This is a contradiction. □

Proof of Theorem 2.1. Half rank implies zero parity and, by (3), $\mathcal{E}(D_s)$. If $m(s) = 0$, Lemma 3.4 contradicts that property. □

Corollary 3.5. *For every $\ell \geq 0$, zero parity and zero first moment imply $2 \operatorname{rank} D_s < 2^\ell$.*

Proof. Square-zero endomorphisms satisfy $2 \operatorname{rank} D_s \leq N$ by image containment in the kernel. Equality is excluded by Lemma 3.4 and (3). □

This is `dyadic_strict_rank_of_zero_moments` in the artifact. It is another formulation of the consequence proved here, not a separate novelty claim.

4 Relation to the published group-algebra criterion

Let k be an algebraic closure of $\mathbb{F}_2$ and $E = (C_2)^\ell$. Denote its group basis by g_x, for $x \in G_\ell$, and put $f = \sum_x s(x) g_x \in kE$. Multiplication by f has matrix D_s in that basis. Set $X_i = g_{e_i} + 1$. The presentation

$$kE \simeq k[X_1, \ldots, X_\ell]/(X_1^2, \ldots, X_\ell^2) \tag{6}$$

is standard for elementary abelian group algebras [7, Section 1.3]. Since $g_x = \prod_i (1 + X_i)^{x_i}$, the constant coefficient of f is $p(s)$ and its linear coefficient vector is $m(s)$.

Friedlander and Pevtsova's flatness criterion states that a map $k[u]/(u^p) \to kE$ is flat precisely when the image of u has a nonzero linear term [5, Example 2.6; author manuscript p. 11]. Benson restates the condition as membership in the augmentation ideal outside its square [7, Section 1.4, p. 6]. Here is the resulting implication for our problem, with the translation made explicit.

If D_s has half rank, then $p(s) = 0$ and $f^2 = 0$, so $u \mapsto f$ defines a map $k[u]/(u^2) \to kE$. A square-zero operator of rank $N/2$ has only size-two Jordan blocks. Consequently kE, as a module over $k[u]/(u^2)$, is a sum of copies of the regular module, hence free and flat. The criterion forces $m(s) \neq 0$. Extending scalars to k changes neither rank nor whether the coefficient vector $m(s)$ vanishes. This proves the implication over $\mathbb{F}_2$.

The erratum to Friedlander–Pevtsova requires the abelian subgroup scheme in a p-point factorization to be unipotent [6]. Here the algebra in (6) is local with nilpotent augmentation ideal, so the elementary abelian case meets that condition. The restriction is therefore respected in this application. These observations explain why the mathematical conclusion is already a consequence of published theory. The translation in this section has not been formalized in our Lean files.

4.1 The descent step as a perturbation specialization

Friedlander–Pevtsova's Proposition 2.2 concerns pairwise commuting endomorphisms α, β, γ in characteristic p, with $\alpha^p = \beta^p = 0$ and $\gamma^{p^r} = 0$. It equates projectivity of the modules on which u acts by α or by $\alpha + \beta\gamma$ [5, Proposition 2.2; author manuscript pp. 8–9].

In our dyadic application, $C^2 = B^2 = 0$ and $BC = CB$. On $V \oplus V$, take

$$\alpha = \begin{pmatrix} C & 0 \\ 0 & C \end{pmatrix}, \quad \beta = \begin{pmatrix} B & 0 \\ 0 & B \end{pmatrix}, \quad \gamma = \begin{pmatrix} 0 & I \\ 0 & 0 \end{pmatrix}. \tag{7}$$

The hypotheses hold with $p = 2$ and $r = 1$, and $\alpha + \beta\gamma = T$. In finite dimension, projectivity for a square-zero action is equivalent to image equaling kernel. Exactness of α is equivalent to exactness of C. Thus the published theorem implies precisely the direction $\mathcal{E}(T) \Rightarrow \mathcal{E}(C)$ used in our induction.

The source proves the more general equivalence using a kernel/image quotient and nilpotence on that quotient. Its calculation applies the preimage property twice. Lemma 3.3 displays the cancellation directly in block coordinates; the dyadic recursion supplies (5). This is a useful change of interface and prerequisites, but it does not establish a new underlying proof principle. The more weakly stated Lean helper need not assume $C^2 = 0$; that hypothesis does hold where it is applied to the dyadic problem.

5 The alternative exact-zero-divisor argument

An independent route uses the necessary direction of Kustin–Şega–Vraciu, Proposition 1.9 [8, Proposition 1.9, p. 6]. For a regular local ring $(Q, \mathfrak{n})$ and $J \subseteq \mathfrak{n}^2$, if the images of $F, G \in Q$ form an exact pair of zero divisors in Q/J, then $FG \notin \mathfrak{n}J$.

To apply it, assume a counterexample has $p(s) = 0 = m(s)$. Use (6) and the regular local presentation

$$Q = k[[X_1, \ldots, X_\ell]], \quad J = (X_1^2, \ldots, X_\ell^2), \quad R = Q/J.$$

Half rank and $f^2 = 0$ imply $\operatorname{ann}_R(f) = (f)$, since multiplication has image equal to kernel. Thus (f, f) is an exact pair. Vanishing constant and linear coefficients allow a lift $F \in \mathfrak{n}^2$. In characteristic

two, F^2 consists of squared monomials of degree at least four; each is divisible by some X_i^2 with a remaining factor of positive degree. Hence $F^2 \in \mathfrak{n}J$, contradicting Proposition 1.9. Its necessary direction has no exclusion of characteristic two.

This route differs substantially from matrix descent: it uses the lift product obstruction in a regular local presentation. The cited proof of Proposition 1.9 invokes a result counting minimal generators via homotopy invariants [8, Theorem 1.8 and Proposition 1.9]. Our complete Lean proof avoids those dependencies. The artifact retains elementary ideal-squaring lemmas in `FrobeniusObstruction.lean`, but neither Proposition 1.9 nor the full algebraic translation is formalized there [11].

6 Lean implementation and reproducibility

The development uses Lean 4 [9] and mathlib [10]. The Lean toolchain is `4.36.0-rc1`. The pinned mathlib revision is

`5f7b4198d22a9c82e1c698a379334fe128095c82`.

The `lean-toolchain`, `lakefile.toml`, and `lake-manifest.json` files record the toolchain and dependency versions. The code and audit snapshot cited here is repository revision

`150bddaab87f80b1d2bf2ee44b8518af7ae2c2b0`.

The manuscript includes a local copy of that development under `artifact/`. Appendix B prints the full inductive source file. Its imports and other definitions are supplied in the same artifact, rather than treated as unproved premises.

File	Role
`Basic.lean`	Signatures, support sums, and source-to-matrix equivalence.
`Structural.lean`	Squaring, parity, rank–nullity reductions, and moments.
`Recursive.lean`	Coordinate slices, triangular descent, induction, and general theorem.
`Checks.lean`	Symbolic consumer examples and declaration axiom reports.

For example, the final signature theorem is assembled as follows:

Listing 3: Assembly of the general theorem from the checked reductions.

```
theorem rankSupportAt_all (ell : ℕ) : RankSupportAt ell := by
  intro s hr hm
  have hp := parity_zero_of_half_rank s hr
  have hs := dyadic_square_zero s hp
  have hc : 2 * (dyadicMatrix s).rank = Fintype.card (Point ell) := by
    rwa [card_point]
  have he := (half_rank_iff_range_eq_kernel (dyadicMatrix s) hs).mp hc
  apply not_kernelInRange_zero_moments ell s hp hm
  intro v hv
  have hv' : v ∈ LinearMap.ker (dyadicMatrix s).mulVecLin := hv
  rw [← he] at hv'
  exact hv'
```

The project also supplies exhaustive Lean checks for small dimensions and a separate Python counterexample search. Neither finite enumeration nor those computational results are premises of the general induction. We make no empirical claim about their search coverage in this paper beyond the records supplied with the artifact.

To reproduce the general check from `artifact/`, install the pinned toolchain and run:

```
lake build
lake env lean Checks.lean
```

Dependency retrieval requires network access or an existing Lake cache. The recorded successful build and the axiom audit are in `verification.log` and `verification.json` [11]. The audit covers 66 declarations, including the general theorem, and reports only `propext`, `Classical.choice`, and `Quot.sound`. There are no unfinished-proof or custom mathematical axioms in those reported dependencies. The symbolic examples quantify over arbitrary dimension; they do not merely instantiate a fixed large matrix.

Kernel checking establishes the formal implication from the definitions used. The mathematical correspondence between those definitions and the intended question still requires review; Sections 2–3 expose it explicitly. The comparisons with published algebra are outside the checked dependency chain.

7 Prior-art search and limits of novelty claims

The bounded formalization search is preserved in the companion audit [11]. It searched 16,263 Lean source files in pinned checkouts of mathlib4, formal-conjectures, TauCeti, and terracini-lean; TauCeti had a recorded sparse scope. Eighteen successful indexed GitHub code queries led to comparison of 23 selected candidate files. Some broad queries reached their 100-file cap. Six rate-limit failures were retained as failures, not as evidence of absence. Exact identifiers were supplemented with algebraic and convolution terminology.

Three native arXiv metadata queries returned 24 records in total; twenty arXiv-restricted indexed web queries supplemented them. Title and abstract screening did not identify a matching formalization. Nearby keyword hits included a Lean formalization concerning dyadic local fields [12] and papers reporting formalizations of Rogers–Ramanujan identities associated with commuting nilpotent matrices [13, 14]. Those are different stated targets; this screening is not a build audit of all their auxiliary code.

The claim supported by these records is that no prior matching Lean statement or complete proof was identified in the searched sources as of October 21, 2026. It is not an exhaustive search of all formal mathematics. Private or unindexed work, historical branches, alternate terminology, and lemmas not advertised in paper metadata can be missed. Priority for the specific inductive exposition is also not determined by this investigation.

The contribution can therefore be described as a completed Lean implementation, a matrix-level exposition, and an explicit translation connecting a coding-theoretic conjecture to a representation-theoretic criterion published approximately 21 years earlier. The chronology motivates the value of this connection, but does not by itself establish novelty or priority for the connection. The descent step specializes Friedlander–Pevtsova's perturbation theorem. Neither the known mathematical conclusion, nor a fundamentally new proof method, nor a first-formalization claim is asserted.

Preparation and attribution. The code and manuscript were developed in an interactive workflow with OpenAI Codex, directed by the project maintainer. The proof was checked with the pinned Lean toolchain; no independent human review is claimed. We credit the dissertation authors and UNL DigitalCommons for the discovery starting points, Gómez-Fonseca, Matthews, Morris, and Pllaha for the explicit question, and the authors of the cited published results for the mathematical principles underlying the literature comparisons. The author line of this review draft is intentionally unset.

References

[1] A. Gómez-Fonseca, G. L. Matthews, K. D. Morris, and T. Pllaha. *Combinatorial Analysis of Dyadic and Quasi-Dyadic Codes.* arXiv:2605.01942v1, posted May 3, 2026. Definition 2.1, Proposition 2.2, and Remark 3.4. https://arxiv.org/html/2605.01942v1.

[2] M. Martinez. *On Dyadic Parity Check Codes and Their Generalizations.* Ph.D. dissertation, University of Nebraska–Lincoln, December 2023. UNL DigitalCommons, Mathematics student research, item 122. https://digitalcommons.unl.edu/mathstudent/122/.

[3] K. Morris. *Analysis of Graph-based Decoders for Quantum Low Density Parity Check Codes.* Ph.D. dissertation, University of Nebraska–Lincoln, August 2025. UNL DigitalCommons, Dissertations and Doctoral Documents, item 339. https://digitalcommons.unl.edu/dissunl/339/.

[4] M. Martinez, T. Pllaha, and C. A. Kelley. Codes based on dyadic matrices and their generalizations. *Advances in Mathematics of Communications* **19**(5) (2025), 1277–1300; online December 13, 2024. https://doi.org/10.3934/amc.2024054.

[5] E. M. Friedlander and J. Pevtsova. Representation-theoretic support spaces for finite group schemes. *American Journal of Mathematics* **127**(2) (April 2005), 379–420. https://doi.org/10.1353/ajm.2005.0010. Locators in this paper refer to Proposition 2.2 and Example 2.6 in the author manuscript, pp. 8–9 and 11: https://sites.math.washington.edu/~julia/Preprints/ppt/ppt.pdf.

[6] E. M. Friedlander and J. Pevtsova. Erratum to "Representation-theoretic support spaces for finite group schemes." *American Journal of Mathematics* **128**(4) (2006), 1067–1068. https://sites.math.washington.edu/~julia/Preprints/ppt/erratum.pdf.

[7] D. J. Benson. *Representations of Elementary Abelian p-Groups and Vector Bundles.* Cambridge Tracts in Mathematics 208, Cambridge University Press, 2016; online 2017. https://doi.org/10.1017/9781316795699. Sections 1.3–1.4, publisher excerpt: https://assets.cambridge.org/97811071/74177/excerpt/9781107174177_excerpt.pdf.

[8] A. R. Kustin, L. M. Şega, and A. Vraciu. *Minimal quasi-complete intersection ideals.* arXiv:1309.1186v2, version posted January 6, 2015. Theorem 1.8 and Proposition 1.9, especially p. 6. https://arxiv.org/pdf/1309.1186v2.

[9] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, 2021, pp. 625–635. https://doi.org/10.1007/978-3-030-79876-5_37.

[10] The mathlib Community. The Lean mathematical library. In *Certified Programs and Proofs (CPP 2020)*, pp. 367–381. https://doi.org/10.1145/3372885.3373824. The exact mathlib4 revision used here is recorded in Section 6.

[11] *Dyadic rank formalization and literature audit*. Companion artifact, October 21, 2026. Repository revision `150bddaab87f80b1d2bf2ee44b8518af7ae2c2b0`, directory `formalization/dyadic-rank`. `https://github.com/alectionary/vibe-math/tree/150bddaab87f80b1d2bf2ee44b8518af7ae2c2b0/formalization/dyadic-rank`. Code, `verification.json`, `verification.log`, `formalization-priority-search.json`, and `arxiv-formalization-search.json` are also supplied with this manuscript (in an arXiv source upload, under `anc/artifact/`); access to the repository is not required to obtain this snapshot.

[12] F. Ueda. *Formalization of Langlands's Second Main Lemma for Local Epsilon Factors*. arXiv:2609.30301v1, 2026. `https://arxiv.org/abs/2609.30301v1`. Cited only as a different formalization found during metadata screening.

[13] Y. Huang, K. Lau, and K. Ono. *Rogers–Ramanujan identities from the geometry of* $X^a = Y^b$. arXiv:2609.20567v1, 2026. Abstract and formalization description. `https://arxiv.org/html/2609.20567v1`.

[14] Y. Huang, K. Lau, K. Ono, and P. Paule. *Algebraic geometric framework of Rogers–Ramanujan identities*. arXiv:2608.15219v1, 2026. Abstract and formalization description. `https://arxiv.org/html/2608.15219v1`.

A Claim and citation guide

Claim or contribution	Evidence and exact scope
Discovery context in UNL dissertations	Martinez [2]; Morris [3]; official DigitalCommons records. Neither is cited as the location of the exact 2026 conjecture.
Explicit source of the converse question	Gómez-Fonseca et al. [1, Remark 3.4].
Antecedent dyadic-code literature	Martinez–Pllaha–Kelley [4], as discussed in [1, Remark 3.4]. No unqualified forward implication is imported.
Established flatness criterion	Friedlander–Pevtsova [5, Example 2.6]; Benson [7, Section 1.4]. Section 4 supplies the translation.
Unipotence condition	Friedlander–Pevtsova erratum [6]; checked for the group algebra in (6).
Published perturbation principle	Friedlander–Pevtsova [5, Proposition 2.2]; substitution (7) is this manuscript's explicit comparison.
Lift product obstruction	Kustin–Şega–Vraciu [8, Proposition 1.9]; Section 5 supplies the application.
General machine-checked implication	Companion code, exact revision, source hashes, build log, and axiom reports [11].
Absence of an identified prior Lean artifact	Bounded search logs and candidate comparisons [11]; this is a report of discovery coverage, not a theorem of absence.

B Complete inductive Lean source

The following is the complete `Recursive.lean` file from the cited artifact. It imports the checked structural results and mathlib. The accompanying `artifact/` directory supplies all local imports, the build configuration, and consumer checks. The listing is generated from that file using TeX escapes for Unicode mathematical glyphs. Line wrapping and glyph rendering do not change the supplied UTF-8 source file.

Listing 4: Complete Recursive.lean, including the general theorem and public matrix interface.

```
module

public import DyadicRank.Structural
public import Mathlib.Data.Fin.Tuple.Basic
public import Mathlib.Tactic.Abel

@[expose] public section

namespace DyadicRank

local instance recursiveFunctionFintype (n : Type) [Fintype n] [DecidableEq n] [BinaryIndex n] :
    Fintype (n → ZMod 2) := binaryFunctionFintype n

/-- Exactness of a square-zero endomorphism, expressed as the required preimage property. -/
def KernelInRange {V : Type*} [AddCommGroup V] [Module F2 V] (f : V →ₗ[F2] V) : Prop :=
  ∀ v, f v = 0 → ∃ w, f w = v

/-- Exactness descends through a square-zero commuting upper triangular extension. -/
theorem kernelInRange_descends {V : Type*} [AddCommGroup V] [Module F2 V]
    (C B : V →ₗ[F2] V) (hc : ∀ v, B (C v) = C (B v)) (hb : ∀ v, B (B v) = 0)
    (h : ∀ v, C v = 0 → ∃ a b, C a + B b = v ∧ C b = 0) : KernelInRange C := by
  intro v hv
  obtain ⟨a, b, hab, hcb⟩ := h v hv
  obtain ⟨c, d, hcd, _⟩ := h b hcb
  have hbb : B b = B (C c) := by
    rw [← hcd, map_add, hb, add_zero]
  refine ⟨a + B c, ?_⟩
  rw [map_add, ← hc, ← hbb]
  exact hab

def sliceZero {ell : ℕ} (v : Point (ell + 1) → F2) : Point ell → F2 :=
  fun x => v (Fin.cons 0 x)

def sliceOne {ell : ℕ} (v : Point (ell + 1) → F2) : Point ell → F2 :=
  fun x => v (Fin.cons 1 x)

def lowerSignature {ell : ℕ} (s : Signature (ell + 1)) : Signature ell :=
  sliceZero s + sliceOne s

/-- Split the first binary coordinate in a sum over all index vectors. -/
theorem sum_split_head {ell : ℕ} (f : Point (ell + 1) → F2) :
    (∑ x, f x) = ∑ y : Point ell, (f (Fin.cons 0 y) + f (Fin.cons 1 y)) := by
  let e : Point (ell + 1) ≃ F2 × Point ell :=
    { toFun := fun x => (x 0, Fin.tail x)
      invFun := fun p => Fin.cons p.1 p.2
      left_inv := Fin.cons_self_tail
      right_inv := by intro p; simp }
  calc
    (∑ x, f x) = ∑ p : F2 × Point ell, f (Fin.cons p.1 p.2) :=
      Fintype.sum_equiv e _ _ (fun x => by simp [e])
    _ = _ := by
      rw [Fintype.sum_prod_type]
      change (∑ a : Fin 2, ∑ y : Point ell, f (Fin.cons a y)) = _
      rw [Fin.sum_univ_two, ← Finset.sum_add_distrib]
      rfl

```

```
@[simp] theorem point_cons_add {ell : ℕ} (a b : F2) (x y : Point ell) :
    (Fin.cons a x : Point (ell + 1)) + Fin.cons b y = Fin.cons (a + b) (x + y) := by
  ext i
  refine Fin.cases ?_ (fun j => ?_) i <;> simp

@[simp] theorem vector_add_self {α : Type*} (v : α → F2) : v + v = 0 := by
  ext x
  exact CharTwo.add_self_eq_zero (v x)

/-- Any two dyadic convolution operators commute. -/
theorem dyadic_mul_comm {ell : ℕ} (s t : Signature ell) :
    dyadicMatrix s * dyadicMatrix t = dyadicMatrix t * dyadicMatrix s := by
  ext x y
  change (∑ z, s (x + z) * t (z + y)) = ∑ z, t (x + z) * s (z + y)
  have h := sum_translate (fun z => s (x + z) * t (z + y)) (x + y)
  simpa [add_left_comm, add_comm, mul_comm] using h.symm

@[simp] theorem dyadicMatrix_add {ell : ℕ} (s t : Signature ell) :
    dyadicMatrix (s + t) = dyadicMatrix s + dyadicMatrix t := rfl

/-- The first moment in the split coordinate is the parity of the one slice. -/
theorem parity_sliceOne {ell : ℕ} (s : Signature (ell + 1)) :
    coefficientParity (sliceOne s) = supportSum s 0 := by
  rw [supportSum_apply, sum_split_head]
  simp [coefficientParity, sliceOne]

theorem parity_lower {ell : ℕ} (s : Signature (ell + 1)) :
    coefficientParity (lowerSignature s) = coefficientParity s := by
  unfold coefficientParity
  rw [sum_split_head]
  rfl

theorem supportSum_lower {ell : ℕ} (s : Signature (ell + 1)) :
    supportSum (lowerSignature s) = Fin.tail (supportSum s) := by
  funext i
  change supportSum (lowerSignature s) i = supportSum s i.succ
  rw [supportSum_apply, supportSum_apply, sum_split_head]
  simp only [lowerSignature, sliceZero, sliceOne, Pi.add_apply, Fin.cons_succ, add_mul]

/-- The usual two-by-two dyadic block recursion, on the zero slice. -/
theorem sliceZero_mulVec {ell : ℕ} (s : Signature (ell + 1)) (v : Point (ell + 1) → F2) :
    sliceZero ((dyadicMatrix s).mulVec v) =
      (dyadicMatrix (sliceZero s)).mulVec (sliceZero v) +
      (dyadicMatrix (sliceOne s)).mulVec (sliceOne v) := by
  funext x
  change (∑ z, s (Fin.cons 0 x + z) * v z) = _
  rw [sum_split_head]
  simp only [point_cons_add, zero_add]
  simp only [sliceZero, sliceOne, Matrix.mulVec, dotProduct, dyadicMatrix,
    Pi.add_apply, Finset.sum_add_distrib]

/-- The usual two-by-two dyadic block recursion, on the one slice. -/
theorem sliceOne_mulVec {ell : ℕ} (s : Signature (ell + 1)) (v : Point (ell + 1) → F2) :
    sliceOne ((dyadicMatrix s).mulVec v) =
      (dyadicMatrix (sliceOne s)).mulVec (sliceZero v) +
      (dyadicMatrix (sliceZero s)).mulVec (sliceOne v) := by
  funext x
  change (∑ z, s (Fin.cons 1 x + z) * v z) = _
  rw [sum_split_head]
  simp only [point_cons_add, add_zero, CharTwo.add_self_eq_zero]
  simp only [sliceZero, sliceOne, Matrix.mulVec, dotProduct, dyadicMatrix,
    Pi.add_apply, Finset.sum_add_distrib]

/-- Duplicate a vector on the two binary slices. -/
def duplicate {ell : ℕ} (v : Point ell → F2) : Point (ell + 1) → F2 :=
  fun x => v (Fin.tail x)

@[simp] theorem sliceZero_duplicate {ell : ℕ} (v : Point ell → F2) :
```

```
    sliceZero (duplicate v) = v := by funext x; simp [sliceZero, duplicate]

@[simp] theorem sliceOne_duplicate {ell : ℕ} (v : Point ell → F2) :
    sliceOne (duplicate v) = v := by funext x; simp [sliceOne, duplicate]

/-- Values on both slices determine the entire vector. -/
theorem slices_ext {ell : ℕ} {v w : Point (ell + 1) → F2}
    (h₀ : sliceZero v = sliceZero w) (h₁ : sliceOne v = sliceOne w) : v = w := by
  have h : ∀ a x, v (Fin.cons a x) = w (Fin.cons a x) := by
    intro a x
    rcases binaryValue a with ha | ha
    · subst a; exact congrFun h₀ x
    · subst a; exact congrFun h₁ x
  funext x
  simpa only [Fin.cons_self_tail] using h (x 0) (Fin.tail x)

@[simp] theorem sliceZero_zero {ell : ℕ} : sliceZero (0 : Point (ell + 1) → F2) = 0 := rfl
@[simp] theorem sliceOne_zero {ell : ℕ} : sliceOne (0 : Point (ell + 1) → F2) = 0 := rfl

/-- The sum of the output slices is the smaller operator on the sum of input slices. -/
theorem sum_slices_mulVec {ell : ℕ} (s : Signature (ell + 1)) (v : Point (ell + 1) → F2) :
    sliceZero ((dyadicMatrix s).mulVec v) + sliceOne ((dyadicMatrix s).mulVec v) =
      (dyadicMatrix (lowerSignature s)).mulVec (sliceZero v + sliceOne v) := by
  rw [sliceZero_mulVec, sliceOne_mulVec]
  simp only [lowerSignature, dyadicMatrix_add, Matrix.add_mulVec, Matrix.mulVec_add]
  abel

/-- Triangular form under the change of coordinates (v₀,v₁) ↦ (v₀,v₀+v₁). -/
theorem triangular_zero_row {ell : ℕ} (s : Signature (ell + 1)) (v : Point (ell + 1) → F2) :
    sliceZero ((dyadicMatrix s).mulVec v) =
      (dyadicMatrix (lowerSignature s)).mulVec (sliceZero v) +
      (dyadicMatrix (sliceOne s)).mulVec (sliceZero v + sliceOne v) := by
  rw [sliceZero_mulVec]
  simp only [lowerSignature, dyadicMatrix_add, Matrix.add_mulVec, Matrix.mulVec_add]
  have h := vector_add_self ((dyadicMatrix (sliceOne s)).mulVec (sliceZero v))
  calc
    _ = (dyadicMatrix (sliceZero s)).mulVec (sliceZero v) +
        ((dyadicMatrix (sliceOne s)).mulVec (sliceZero v) +
         (dyadicMatrix (sliceOne s)).mulVec (sliceZero v)) +
        (dyadicMatrix (sliceOne s)).mulVec (sliceOne v) := by rw [h]; simp only [add_zero]
    _ = _ := by abel

/-- A cycle for the smaller operator duplicates to a cycle for the larger one. -/
theorem duplicate_cycle {ell : ℕ} (s : Signature (ell + 1)) (v : Point ell → F2)
    (hv : (dyadicMatrix (lowerSignature s)).mulVec v = 0) :
    (dyadicMatrix s).mulVec (duplicate v) = 0 := by
  apply slices_ext
  · rw [sliceZero_mulVec, sliceZero_duplicate, sliceOne_duplicate]
    have h := hv
    simpa only [lowerSignature, dyadicMatrix_add, Matrix.add_mulVec, sliceZero_zero, sliceOne_zero] using h
  · rw [sliceOne_mulVec, sliceZero_duplicate, sliceOne_duplicate, add_comm]
    have h := hv
    simpa only [lowerSignature, dyadicMatrix_add, Matrix.add_mulVec, sliceZero_zero, sliceOne_zero] using h

/-- Vanishing of the first support moment makes exactness descend one dimension. -/
theorem kernelInRange_lower {ell : ℕ} (s : Signature (ell + 1))
    (hm : supportSum s = 0) (he : KernelInRange (dyadicMatrix s).mulVecLin) :
    KernelInRange (dyadicMatrix (lowerSignature s)).mulVecLin := by
  have hp : coefficientParity (sliceOne s) = 0 := by rw [parity_sliceOne, hm]; rfl
  apply kernelInRange_descends (dyadicMatrix (lowerSignature s)).mulVecLin
    (dyadicMatrix (sliceOne s)).mulVecLin
  · intro v
    simp only [Matrix.mulVecLin_apply]
    rw [Matrix.mulVec_mulVec, Matrix.mulVec_mulVec, dyadic_mul_comm]
  · intro v
    simp only [Matrix.mulVecLin_apply]
    rw [Matrix.mulVec_mulVec, dyadic_square_zero _ hp]
    simp
```

```
  · intro v hv
    obtain ⟨w, hw⟩ := he (duplicate v) (by
      simpa only [Matrix.mulVecLin_apply] using duplicate_cycle s v (by
        simpa only [Matrix.mulVecLin_apply] using hv))
    simp only [Matrix.mulVecLin_apply] at hw
    refine ⟨sliceZero w, sliceZero w + sliceOne w, ?_, ?_⟩
    · have h := congrArg sliceZero hw
      rw [triangular_zero_row, sliceZero_duplicate] at h
      simpa only [Matrix.mulVecLin_apply] using h
    · have h := congrArg (fun u => sliceZero u + sliceOne u) hw
      rw [sum_slices_mulVec, sliceZero_duplicate, sliceOne_duplicate, vector_add_self] at h
      simpa only [Matrix.mulVecLin_apply] using h

/-- A zero-parity signature with zero first moments cannot be exact in any dimension. -/
theorem not_kernelInRange_zero_moments (ell : ℕ) :
    ∀ s : Signature ell, coefficientParity s = 0 → supportSum s = 0 →
      ¬ KernelInRange (dyadicMatrix s).mulVecLin := by
  induction ell with
  | zero =>
    intro s hp _ he
    have hs : ∀ x : Point 0, s x = 0 := by
      intro x
      have hu : (Finset.univ : Finset (Point 0)) = {x} := by
        ext y
        simp only [Finset.mem_univ, Finset.mem_singleton, true_iff]
        exact Subsingleton.elim y x
      unfold coefficientParity at hp
      rwa [hu, Finset.sum_singleton] at hp
    have hm : dyadicMatrix s = 0 := by
      ext x y
      exact hs (x + y)
    have hz : (dyadicMatrix s).mulVecLin = 0 := by rw [hm, Matrix.mulVecLin_zero]
    obtain ⟨w, hw⟩ := he (fun _ => 1) (by rw [hz]; rfl)
    have h := congrFun hw 0
    rw [hz] at h
    norm_num at h
  | succ ell ih =>
    intro s hp hm he
    apply ih (lowerSignature s)
    · rwa [parity_lower]
    · rw [supportSum_lower, hm]
      rfl
    · exact kernelInRange_lower s hm he

/-- The full rank/support converse, for every binary dimension. -/
theorem rankSupportAt_all (ell : ℕ) : RankSupportAt ell := by
  intro s hr hm
  have hp := parity_zero_of_half_rank s hr
  have hs := dyadic_square_zero s hp
  have hc : 2 * (dyadicMatrix s).rank = Fintype.card (Point ell) := by
    rwa [card_point]
  have he := (half_rank_iff_range_eq_kernel (dyadicMatrix s) hs).mp hc
  apply not_kernelInRange_zero_moments ell s hp hm
  intro v hv
  have hv' : v ∈ LinearMap.ker (dyadicMatrix s).mulVecLin := hv
  rw [← he] at hv'
  exact hv'

/-- The unrestricted positive-dimensional conjecture is proved. -/
theorem rankSupportConjecture_proved : RankSupportConjecture := by
  intro ell _
  exact rankSupportAt_all ell

/-- The strict rank deficit is now proved in every dimension. -/
theorem dyadic_strict_rank_of_zero_moments {ell : ℕ} (s : Signature ell)
    (hp : coefficientParity s = 0) (hm : supportSum s = 0) :
    2 * (dyadicMatrix s).rank < 2 ^ ell :=
  (rankSupportAt_iff_strict_deficit ell).mp (rankSupportAt_all ell) s hp hm
```

```
261
262 /-- The theorem for any dyadic matrix and any caller's finite enumeration. -/
263 theorem dyadic_rank_support {ell : ℕ} [inst : Fintype (Point ell)]
264     (D : Matrix (Point ell) (Point ell) F2) (hd : IsDyadic D)
265     (hr : 2 * D.rank = 2 ^ ell) : supportSum (D 0) ≠ 0 := by
266   apply (rankSupportAt_iff ell).mp (rankSupportAt_all ell) D hd
267   have hi : binaryFunctionFintype (Fin ell) = inst := Subsingleton.elim _ _
268   change 2 * @Matrix.rank (Point ell) (Point ell) F2
269     (binaryFunctionFintype (Fin ell)) _ D = 2 ^ ell
270   rw [hi]
271   exact hr
272
273 /-- The source's half-rank form, including the positive-dimension hypothesis. -/
274 theorem dyadic_rank_support_half {ell : ℕ} [Fintype (Point ell)]
275     (hpos : 0 < ell) (D : Matrix (Point ell) (Point ell) F2) (hd : IsDyadic D)
276     (hr : D.rank = 2 ^ ell / 2) : supportSum (D 0) ≠ 0 := by
277   apply dyadic_rank_support D hd
278   cases ell with
279   | zero => omega
280   | succ k =>
281     rw [hr, pow_succ]
282     omega
283
284 end DyadicRank
```